ARM PrimeCell Driver

CLCD v1.0 (PL110)

Test Report

PL110-TREP-1000-A

Open Access

Proprietary notice

Document confidentiality status

This document is Open Access. This document has no restriction on distribution.

Product status

The information in this document is Final (information on a developed product).

Web address

http://www.arm.com

Feedback

ARM limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Document Summary

Document Issue	A
PrimeCell driver identifier	CLCD
Code Version	1.0
PrimeCell identifier	PL110
Test date	28/02/01

Contents

1 ABOUT THIS DOCUMENT

1.1 Change history

Issue	Date	Change
A	27/2/2001	First release of test report

2 RELEASED FILES

PL110-ISPC-1000	API document
PL110–TREP-1000	Test Results document
apclcd/apclcd.h	Public header file
apclcd/clcd.h	Private header file
apclcd/clcd.c	Source code
apclcd/clcdtst.c	Self-test module
apclcd /draw.h	Drawing functions header file
apclcd /apdraw.c	Drawing functions source module
apclcd /clr8x8.c	Test font for drawing functions
apclcd /clr8x8.h	Test font declaration

3 TEST ENVIRONMENT

3.1 Test Specification Summary

	Specification (Intended)	Actual Test Environment If different from Specification
Code Test Platform	INTEGRATOR	
Core used for test	ARM7TDMI	
Module ROM size (RO code + RO data)	2968 Bytes	
Module RAM size (ZI data) per instance	56 Bytes	
Primecell tested	PL110	
External hardware used	Logic Module with IM-PD1 board. Fitted with a suitable LCD such as the Citizen K3240H-FR STN.	
Performance obtained	N/A	
Driver modules	clcd.c	
Test modules	clcdtst.c, apdraw.c clr8x8.c	
Standard modules required (normal archive location)	os.c, oss.s, int.c, dispatch.s, boot.s, vectors.s, test.c, main.c, timer.c	
Custom modules required (in [project]/privates/ apCLCD)	applatfm.h	

3.2 Hardware requirements

3.2.1 FPGA

Is the image used to program the FPGA archived	NO
If so, at what location	N/A

3.2.2 Test configuration

Is the hardware configuration documented	NO
If so, at what location	N/A

3.2.3 External connectors required

N/A

3.2.4 Devices on prototyping area

None

3.2.5 External test equipment

PD1 board fitted with a suitable LCD

4 CODE COVERAGE

Code coverage tests have been performed. YES

Code coverage details below

```
Code Coverage report
======================

Normal Routine Exit:
Missed: CC(X,8) CC(X,26)
Code coverage: 92 %

Routine returns:
Missed: CC(R,0) CC(R,1) CC(R,2) CC(R,3) CC(R,4) CC(R,5) CC(R,6) CC(R,7)
Code coverage: 0 %

Blocks:
Missed:
CC(B,6) CC(B,7) CC(B,8) CC(B,10) CC(B,11) CC(B,12) CC(B,15) CC(B,16)
CC(B,17) CC(B,19) CC(B,21) CC(B,22) CC(B,23) CC(B,24) CC(B,25) CC(B,26)
CC(B,27) CC(B,29) CC(B,30) CC(B,35) CC(B,36) CC(B,39) CC(B,42) CC(B,46)
Code coverage: 48 %
```

The following results were obtained with all extended tests:

```
Code Coverage report
======================

Normal Routine Exit:
Missed: Code coverage: 100 %

Routine returns:
Missed: CC(R,0) CC(R,1) CC(R,2) CC(R,3) CC(R,4) CC(R,5) CC(R,6) CC(R,7) CC(R,8)
Code coverage: 0 %

Blocks:
Missed: CC(B,6) CC(B,7) CC(B,8) CC(B,10) CC(B,11) CC(B,12) CC(B,15) CC(B,16) CC(B,19)
CC(B,25) CC(B,26) CC(B,27) CC(B,30) CC(B,35) CC(B,39) CC(B,46)
Code coverage: 65 %
```

Percentage of normal routine exits tested. 100 %

- Examine all routine exits listed as untested to ensure that these are not used in normal flow.

All untested routine exits examined. YES

Percentage of blocks tested. 65 %

4.1 Untested Returns

Returns not covered by these tests have been examined and are all only taken if an invalid set of parameters is selected.

4.2 Untested Blocks

Blocks not covered by these tests have been examined. They would only be entered for certain combinations of settings or as a result of external error condition. The settings are not applicable to the LCDs available for test. The conditions under which they would be entered are categorised as below.

4.2.1 Untested Block Usage

Usage	Block numbers
Used for 8-bit monochrome displays	CC(B,6) CC(B,10) CC(B,15)
Used for 4-bit monochrome displays	CC(B,7) CC(B,11) CC(B,16)
Avoiding invalid settings	CC(B,8) CC(B,12) CC(B,19)
Dual-panel displays only	CC(B,17) CC(B,21) CC(B,22) CC(B,23) CC(B,24) CC(B,25) CC(B,26) CC(B,27) CC(B,2)8 CC(B,29) CC(B,42) CC(B,46)
BGR colour format	CC(B,30)
Odd number of palette entries	CC(B,35)
FIFO Overflow error	CC(B,36)
AHB Bus Master error	CC(B,39)

5 RELEASED TEST MODULE

5.1 Overview

The released test module is named **`CLCDtst.c`**.

It is used as a self-test module for the **`CLCD`** PrimeCell and also as a part of the formal testing.

Test environment is as described earlier.	YES
Code coverage: Percentage of blocks tested by this test module.	65 %

5.2 Changes to test environment

None

5.3 Outline

The test module performs the following functions:

5.3.1 Initialisation

The following API calls are made to set up the device:

- ⊕ Call to `apCLCD_Initialize()` function to set up peripheral base address and interrupts.
  - Number of interrupts used 1
  - Specific parameters passed in configuration data NONE
- ⊕ Call to CLCD_PanelInitialize() function to set up the major display parameters and configuration and start the LCD.
  - Specific parameters are passed as a pointer to the configuration structure. For the Citizen K3240H-FR the initialisation is:

Structure element	Setting used	Description
eType	apCLCD_STN	LCD display type
eColorOrder	apCLCD_RGB	RGB or BGR
eReload	apCLCD_RELOAD_EARLY	FIFO reload timing
eClockSource	apCLCD_INTERNAL_CLOCK	LCD clock source
ACBias	4	Number of line clock periods between each toggle of the AC-bias pin
eVSyncActive	apCLCD_LCDFP_H	LcdFP pin active state
eHSyncActive	apCLCD_LCDLP_H	LcdLP pin active state
eDataDrive	apCLCD_CLCP_R	edge data driven on
eTftClac	apCLCD_CLAC_H	CLAC pin setting in TFT mode
ClleDelay	0	Delay before line end pulse
eColorType	apCLCD_STN_COLOR	STN LCD color type

eInterface	apCLCD_STN_8BIT	STN LCD x-bit interface
ePanel	apCLCD_STN_SINGLE	STN LCD panel type
Clock	LCD_CLOCK_SPEED	LCD controller clock in Hz
Vsync	1	vertical sync pulse width
VFront	0	vertical front porch
Vback	0	vertical back porch
Hsync	20	horizontal sync pulse width
HFront	20	horizontal front porch
Hback	20	horizontal back porch
Refresh	FRAME_RATE	LCD screen refresh rate in Hz
eVCInterTime	apCLCD_VSYNC	LCD VComp interrupt timing
DMABase	CLCD_SCREEN_BASE	Base address of screen memory
LineWidth	SCREEN_WIDTH	line length, multiple of 16 pixels
NumLines	SCREEN_HEIGHT	total height of the screen
ePixelOrder	apCLCD_LITTLE_ENDIAN_BYTES	pixel ordering within a byte
eBpp	apCLCD_4BPP	bits per pixel
PwerUpDel	POWER_UP_DEL	Delay between LcdEn & LcdPwr on
PwerDownDel	POWER_DOWN_DEL	Delay between LcdPwr & LcdEn off
rPreamble	CitizenPreamble	Called right at the start
rSwitchOff	CitizenSwitchOff	Special sequence for power off
rSwitchOn	CitizenSwitchOn	Special sequence for power on

- For the Sharp LQ084V1DG21 the initialisation is:

Structure element	Setting used	Description
eType	apCLCD_TFT	LCD display type
eColorOrder	apCLCD_RGB	RGB or BGR
eReload	apCLCD_RELOAD_EARLY	FIFO reload timing
eClockSource	apCLCD_INTERNAL_CLOCK	LCD clock source
ACBias	0	Number of line clock periods between each toggle of the AC-bias pin
eVSyncActive	apCLCD_LCDFP_H	LcdFP pin active state
eHSyncActive	apCLCD_LCDLP_H	LcdLP pin active state
eDataDrive	apCLCD_CLCP_R	edge data driven on
eTftClac	apCLCD_CLAC_H	CLAC pin setting in TFT mode
ClleDelay	0	Delay before line end pulse
eColorType	apCLCD_STN_COLOR	STN LCD color type
eInterface	apCLCD_STN_4BIT	STN LCD x-bit interface
ePanel	apCLCD_STN_SINGLE	STN LCD panel type
Clock	LCD_CLOCK_SPEED	LCD controller clock in Hz
Vsync	25	vertical sync pulse width

VFront	7	vertical front porch
Vback	36	vertical back porch
Hsync	64	horizontal sync pulse width
HFront	65	horizontal front porch
Hback	33	horizontal back porch
Refresh	FRAME_RATE	LCD screen refresh rate in Hz
eVCInterTime	apCLCD_VSYNC	LCD VComp interrupt timing
DMABase	CLCD_SCREEN_BASE	Base address of screen memory
LineWidth	SCREEN_WIDTH	line length, multiple of 16 pixels
NumLines	SCREEN_HEIGHT	total height of the screen
ePixelOrder	apCLCD_LITTLE_ENDIAN_BYTES	pixel ordering within a byte
eBpp	apCLCD_8BPP	bits per pixel
PwerUpDel	POWER_UP_DEL	Delay between LcdEn & LcdPwr on
PwerDownDel	POWER_DOWN_DEL	Delay between LcdPwr & LcdEn off
rPreamble	SharpPreamble	Called right at the start
rSwitchOff	SharpSwitchOff	Special sequence for power off
rSwitchOn	SharpSwitchOn	Special sequence for power on

⊕ Call to apCLCD_RegisterCallback() function to register the test program's local interrupt handler. This counts Vertical Compare and Base Address Update interrupts and reports FIFO underflow and AHB Bus Master Error interrupts.

5.3.2 Interrupt Tests

The CLCD is configured into a minimal setting with 1 bit per pixel and an appropriate palette using the following API calls:

⊕ Call to apCLCD_BppSet() to select 1 bit per pixel

⊕ Call to apCLCD_PaletteSet() to initialise a black/white two entry palette

⊕ Call to apCLCD_PowerUp() to ensure the PrimeCell is running

⊕ Call to apCLCD_InterruptsDisable() to ensure there are no pending interrupts

With this basic configuration the display is cleared to black and the Base Address Update and Vertical Compare interrupts are tested in turn. The interrupt under test is enabled and counted for a period of three seconds. This is merely to establish whether interrupts are happening. No attempt is made to measure frequency etc.

If Base address interrupts are enabled they are expected to happen on every display update. The Vertical Compare event interrupts are always expected on start of active area and start of Vertical Sync. They normally happen on start of front or back porch too, but not if the corresponding porch setting is zero.

⊕ Call to apCLCD_InterruptEventsEnable() for the Base Address Update and Vertical Compare in turn as they are tested.

⊕ Call to apCLCD_VCompEventsEnable() to select the Vertical Compare interrupt events as needed

Once the tests are complete the Base Address Update and Vertical Compare interrupts are disabled, and the FIFO underflow and AHB Bus Master Error are enabled. If the latter two happen at any point later in the test they are reported on the console.

Example: Interrupt test display:

```
Testing Base address interrupts
Base address interrupts OK

Testing Vertical Compare interrupts
Vertical sync: interrupts expected
Interrupts as expected
Vertical Sync interrupts OK
Back porch: no interrupts expected
Interrupts as expected
Back Porch interrupts OK
Active video: interrupts expected
Interrupts as expected
Active video interrupts OK
Front porch: no interrupts expected
Interrupts as expected
Front Porch interrupts OK
Vertical Compare interrupt tests complete
```

- Call to apCLCD_InterruptEventsEnable() to enable FIFO underflow and AHB Bus Master Error. These remain enabled for the rest of the test. If they occur these errors are reported.

5.3.3 Refresh Rate Tests

The refresh rate of the display is determined by the frequency of the Clock supplied to the PrimeCell and a divisor applied to this clock to generate the panel clock. The refresh rate can be established by calculating the divisor. The CLCD driver does this, and programs the divisor appropriately. Since this is set to the nearest integral number, the resulting refresh rate set up is likely to differ from that requested. For high refresh rates this difference may be large.

The driver is configured for a sequence of rates so that their effect on display quality can be observed. Each setting is held for three seconds and the number of Vertical Compare events counted. The setting requested, the setting established and the measured refresh rates are displayed.

- Call to apCLCD_InterruptEventsEnable() to enable the Vertical Compare interrupt
- Call to apCLCD_VCompEventsEnable() to select Vertical sync events
- Call to pCLCD_RefreshSet() to set the refresh rate to test

Refresh rate test display example:

```
Check a range of refresh rates

Nominal:  5
Actual:   5
Measured: 11

Nominal:  25
Actual:   25
Measured: 15
```

```
Nominal:  45
Actual:   48
Measured: 29

Nominal:  65
Actual:   72
Measured: 44

Nominal:  85
Actual:   87
Measured: 52
```

5.3.4 Display Tests

The display is configured for a range of bits per pixel in turn, and for each setting a suitable palette is generated. For 1 and 2 BPP the palette is monochrome, and for higher values the palettes are colour. The 4 bit per pixel palette is generated for a known sequence of colours, which are reported as they are generated. The display is cleared to a series of single colours, and the palette entry number is reported as the test proceeds. For 16 and 24 BPP settings, the values are written directly to the LCD.

- ⊕ Call to apCLCD_BppSet() to select bits per pixel
- ⊕ Call to apCLCD_PaletteSet() to initialise the appropriate palette
- ⊕ Write the screen to a series of single colours

5.4 Results

Test run returns TRUE	YES
When apTEST_INTERACTIVE is FALSE, no user interaction is required	YES
When apTEST_VERBOSE is FALSE, no output is produced	NO

6 FURTHER TESTS

6.1 Overview

The PL110 PrimeCell has a very large number of options, which results in a large number of possible paths through the driver code. Ideally the tests should be repeated for a range of LCDs of different types so that all paths are tested. However this is not normally possible in practice. Limited tests further can be done if the options are modified and the test repeated. Care must be taken not to damage the LCD in use by setting inappropriate options, and it is safest to disconnect the LCD from the board for some tests. The display cannot be inspected, but all interrupt and refresh tests are valid, and it ensures that there are no major faults in the code.

During development of the software the tests were run satisfactorily on a Citizen K3240H-FR STN display, Sharp LQ084V1DG21 and Toshiba LTM10C209H TFT displays.

6.1.1 Outline

A number of options and associated code paths are unused for the above displays. The majority of these are for dual panel LCDs. An associated feature that is untested is the mapping of two contiguous areas of memory onto the LCD. As additional tests the options and code can be modified to exercise these parts of the code. Apart from the limited tests above there is no other major configurable option that would significantly expand the test

6.2 Dual Panel

The PL110 PrimeCell has a very large number of options, which results in a large number of possible paths through the code. The results from the code coverage macros were inspected, and the major single parameter that affects the coverage is dual panel STN display. The configuration was changed to this setting and re-run. Contiguous Memory Regions

Since it is expected that the two panel's memory regions are contiguous, mapping as either single or dual panel should result in the same display. The driver code was modified to check the mapping. The display parameters were changed so that the single-panel Citizen panel under test was said to be a dual panel. The driver code was then modified so that the PrimeCell registers were all set up as single panel, but all mapping code was unchanged. This causes the calculation of bytes per panel to be done, and the mapping dependant parameters to be returned to drawing functions as dual panel. The resulting display should be the same if all mapping is correct. This was found to be so.

6.3 Different Display Types

To perform limited further testing the configuration was changed to settings suitable for a Toshiba LTM10C209H TFT display. This display was not available, so the Citizen LCD was disconnected from the PD1 board, which was retained with the Logic module. The driver was then run without risk of damaging a display. This test ensured that there was no major failure, and that interrupts and clock signals were generated, but could not establish that the display was working. This expanded the code coverage by 8%, but to take this any further was not feasible.

6.4 Results of Extended Tests

By running the extended tests the code coverage increased, with normal routine exits at 100%, and blocks entered increased from 48% to 65% of blocks.